

EXPERIMENT 3

By Lakshya Yadav 590034429

Aim: To understand common C errors, GNU/GDB, Linux shell and kernel, and important GDB debugging points.

1. Error Classification

Below are small C examples for each error type. I compiled/ran each example to observe what happens.

A. Syntax Error

```
#include <stdio.h>
int main()
{
    int a = 10
    printf("%d\n", a);
    return 0;
}
```

Terminal

```
$ gcc syntax.c -o syntax
syntax.c: In function 'main':
error: expected ',' or ';' before 'printf'
```

Reason: the semicolon after `int a = 10` is missing.

B. Compilation / Type Error

```
#include <stdio.h>
int main()
{
    int *p = "hello";
    printf("%d\n", *p);
    return 0;
}
```

Terminal

```
$ gcc -Wall -Wextra type.c -o type
warning: initialization of 'int *' from incompatible pointer type
```

Reason: a string address (`char *`) is assigned to an `int` pointer.

C. Linker Error

```
#include <stdio.h>
void showMessage();
int main()
{
    showMessage();
    return 0;
}
```

Terminal

```
$ gcc linker.c -o linker
undefined reference to `showMessage'
collect2: error: ld returned 1 exit status
```

Reason: showMessage() is declared and called, but its function definition is missing.

D. Runtime Error

```
#include <stdio.h>
int main()
{
    int a = 10, b = 0;
    printf("%d\n", a / b);
    return 0;
}
```

Terminal

```
$ gcc -g runtime.c -o runtime
$ ./runtime
Floating point exception (core dumped)
```

Reason: division by zero happens while the program is running.

E. Logical Error

```
#include <stdio.h>
int main()
{
    int a = 8, b = 3;
    int sum = a - b;    // should be a + b
    printf("Sum = %d\n", sum);
    return 0;
}
```

Terminal

```
$ gcc logical.c -o logical
$ ./logical
Sum = 5
```

Expected result is 11, but output is 5. The program compiles because a - b is valid C syntax; the logic is wrong.

2. GNU Concept

GNU stands for "GNU's Not Unix!". It is a free software project started by Richard Stallman in 1983 with the goal of creating a free Unix-like operating system. Many tools used on Linux systems, such as GCC and GDB, are part of the GNU software ecosystem.

GDB (GNU Debugger) is the debugger used to inspect a program while it is running. It can stop execution at selected lines, show variable values, move through code one line at a time, and help find crashes. A C program is normally compiled with the **-g** option so GDB has debugging information.

Example GDB session

```
$ gcc -g -Wall -Wextra program.c -o program
$ gdb ./program
GNU gdb ...
(gdb) break main
(gdb) run
```

3. Shell Context

Shell: A shell is the command-line program through which a user gives commands to the operating system. It reads commands such as ls, cd, gcc and gdb and asks the operating system to perform the required work.

Kernel: The kernel is the core part of the operating system. It manages hardware, memory, processes, files and communication between software and hardware.

Three common Linux shells:

Shell	Short note
bash	Bourne Again Shell; very common on Linux.
zsh	Feature-rich shell with strong interactive features.
fish	Friendly Interactive Shell; designed to be easy to use.

4. Breakpoint, Watchpoint, Catchpoint and Tracepoint

These are different ways of telling a debugger what event or location is important during program execution.

Type	What it does	Simple example
Breakpoint	Stops program execution when a chosen source line or function is reached.	break main break 12
Watchpoint	Stops when the value of a selected variable or memory expression changes.	watch total
Catchpoint	Stops when a selected event occurs, such as a signal, exception, fork or system call (depending on target/support).	catch signal SIGSEGV
Tracepoint	Collects debugging data at a location without the normal stop-and-inspect workflow; mainly used with GDB trace experiments/remote targets.	trace function_name

In short: breakpoint = stop at a place; watchpoint = stop when data changes; catchpoint = stop when an event happens; tracepoint = collect information for later analysis.

Conclusion

From these exercises I understood that not every C error is found in the same stage. Compiler messages and warning flags help with syntax/type problems, while GDB is useful for checking program state, logic and runtime failures. I also learned the basic role of the shell, kernel and different debugging points.

Note: Terminal panels above show representative command/output formatting for the exercise. Actual addresses and exact diagnostic wording can differ by GCC/GDB version and Linux system.